

Therayu

AI Physiotherapy Platform

Flutter Application — Technical Report

Architecture, implementation, dependencies and performance engineering

Framework	Flutter 3.47.0 (stable) · Dart 3.13.0
Targets	Web (Chrome) · Android (arm64)
Dart source	18 files · 6,037 lines · 76 classes · 5 enums
Runtime packages	6 direct dependencies
Backend	Python FastAPI WebSocket server, 727 lines
Clinical engine	Pre-existing, unmodified — 5 modules imported



1. Executive summary

Therayu began as a Python desktop application: a webcam window driven by `live_pose.py`, rendering a skeleton and clinical joint angles through OpenCV. This report documents its conversion into a Flutter application with a designed interface, and the engineering decisions that conversion required.

The central constraint shaped everything else. The clinical engine is Python: MediaPipe pose and hand detection, sub-pixel refinement, confidence calibration, anatomical validation, One-Euro temporal filtering, and a goniometric angle engine implementing standard physiotherapy conventions. Reimplementing that in Dart would have meant re-deriving validated clinical mathematics and revalidating every angle. Instead the engine was left untouched and wrapped in a WebSocket server; Flutter became a real-time client that captures frames, renders the skeleton, and presents the readouts.

The finding that made this viable

The trained model file `best_upper_body_model.pkl` is **not part of the live pipeline**. It is loaded only by `trainer.py`, `evaluator.py` and `inference.py` — the offline, static-image path. `live_pose.py` never touches it. The live path is MediaPipe plus deterministic geometry, which meant the conversion carried no risk to the trained classifier whatsoever.

What was delivered

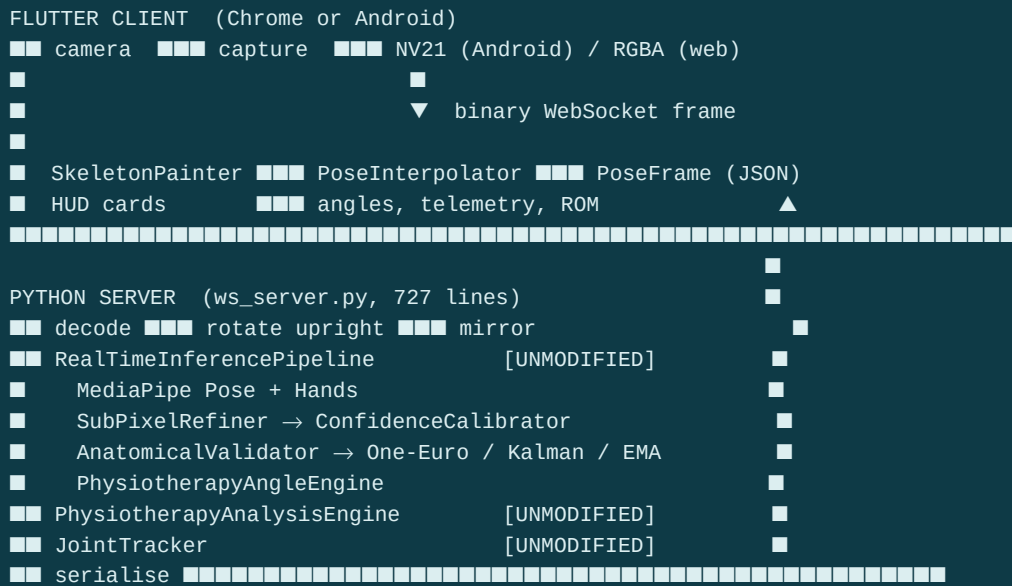
- A Flutter application of 6,037 lines across 18 Dart files, running on Chrome and Android from one codebase.
- A 727-line Python WebSocket server that imports the existing clinical modules and calls them in the same order as the desktop application's inference thread.
- A design system implementing the Figma brand, with every colour, radius and text style in a single file.
- Two capture paths tuned per platform: raw YUV420 to NV21 on Android, direct canvas capture on web.
- A latency investigation that reduced skeleton lag from several seconds to approximately one frame.

Clinical fidelity

Every angle the application displays is computed by the original Python engine. Landmark names, visibility thresholds, normal ranges and the three distinct 'no value' states were verified programmatically against the Python source: all 23 body landmark references, all 21 hand joints and all 17 consumed angle keys matched exactly, with no unknown names and no keys consumed that the engine does not produce.

2. System architecture

The system is two processes joined by a WebSocket. Flutter owns the camera and everything visible; Python owns everything clinical. Neither reaches into the other's territory.



Why frames travel to the server

Three alternatives were considered. Porting the engine to Dart with ML Kit would reimplement validated clinical mathematics. Streaming server-rendered video would keep the OpenCV drawing and forfeit a native interface. Streaming pixels one way and landmarks back keeps the validated engine authoritative while letting Flutter render natively at display rate. The third was chosen.

Server modules imported, not rewritten

Module	Source file	Responsibility
RealTimeInferencePipeline	<code>inference/pipeline.py</code>	MediaPipe detection, refinement, calibration, validation, filtering, joint angles
PhysiotherapyAnalysisEngine	<code>src/physio_analysis.py</code>	Range-of-motion accumulation, movement quality, clinical feedback
JointTracker	<code>tracking/tracker.py</code>	Persistent track identifiers across frames
MedicalGUIRenderer	<code>visualization/medical_gui.py</code>	Server-side annotated capture for screenshots and recordings
SessionLogger	<code>utilities/logger.py</code>	MP4 and PNG output

3. Flutter and Dart stack

Toolchain

Component	Version	Note
Flutter SDK	3.47.0 stable	Installed automatically by setup_flutter.ps1
Dart SDK	3.13.0	Bundled with Flutter
Dart constraint	>=3.12.0 <4.0.0	From pubspec.lock
Flutter constraint	>=3.44.0	From pubspec.lock
DevTools	2.60.0	Profiler and inspector
Linters	flutter_lints 4.0.0	Analyzer reports zero issues

Runtime dependencies

Six direct packages. Nothing was added that the application does not use on every session, and no state-management framework was introduced — a single `ChangeNotifier` is sufficient for one screen with one stream of truth.

Package	Pinned	Purpose in this application
camera	0.11.4	Device camera. Android uses camera_android_camerax 0.6.30; web uses camera_web 0.3.5+5
web_socket_channel	3.0.3	Binary frame upload and JSON result stream; browser WebSocket on web
http	1.6.0	Pre-flight /health check before opening the socket
shared_preferences	2.5.5	Persists server address, body mode, mirror and quality; localStorage on web
wake_lock_plus	1.7.0	Keeps the screen awake mid-session
web	1.1.1	Canvas interop for the fast web capture path (see section 6)

Dart language features relied on

- **Sealed classes and exhaustive pattern matching** — `AngleState` is sealed with four variants, so the compiler guarantees every rendering path handles all of them.
- **Records** — `{bool ok, String detail}` returns the health-check result without a single-use class.
- **Switch expressions with patterns** — used for mode mapping, socket status colours and corner assignment.
- **Conditional imports** — `if (dart.library.js_interop)` selects the real canvas grabber on web and a stub on Android, so web-only APIs never reach the Android build.
- **JS interop** — `dart:js_interop` with `package:web` for direct DOM canvas access.
- **Extension types and null safety** throughout; the analyzer runs clean.

4. Source code map

Every Dart file, its size, and what it is responsible for. Comment density is roughly 16% overall and considerably higher in the files where a future reader would otherwise have to reverse-engineer a decision.

Totals	Dart files	Lines	Code	Comments	Classes	Enums
lib/	18	6,037	4,319	949 (16%)	76	5
test/	1	82	—	—	—	—

Entry point and screens

File	Lines	Contents
main.dart	64	App entry, portrait lock, system UI styling, theme installation
screens/splash_screen.dart	118	Brand splash; also warms the camera permission so the next screen opens live
screens/connect_screen.dart	396	Body-mode picker, server address, pre-flight health check, setup guidance
screens/live_session_screen.dart	577	The session: full-bleed camera, skeleton overlay, four-corner HUD, display-rate ticker

Services — the moving parts

File	Lines	Contents
services/session_controller.dart	360	The single ChangeNotifier the UI listens to; owns socket, camera and interpolator
services/pose_socket.dart	370	WebSocket transport, binary framing, backpressure, exponential-backoff reconnect
services/camera_streamer.dart	566	Camera lifecycle and both capture paths, including the stride-aware NV21 repack
services/pose_interpolator.dart	144	Lerps between landmark sets so motion is smooth at display rate
services/frame_grabber_web.dart	126	Direct canvas capture on web — the fix for multi-second lag
services/frame_grabber_stub.dart	31	Matching no-op API so the Android build never sees web-only imports

Models, theme and widgets

File	Lines	Contents
models/pose_frame.dart	347	Wire models; defensive parsing of the three-state angle contract
models/skeleton_topology.dart	305	Bone tables, goniometric arc definitions, HUD groups, visibility floors
theme/app_theme.dart	532	The complete design system — the only file containing a colour literal
widgets/skeletonPainter.dart	515	Skeleton, arcs, angle badges, framing brackets, quality ring
widgets/status_widgets.dart	647	Telemetry bar, guidance banners, assessment panel, feedback bar, mode switcher
widgets/controls_sheet.dart	387	Session controls replacing the desktop hotkeys
widgets/brand.dart	384	Gold wave, watermark, wordmark, logo tile
widgets/angle_card.dart	168	One clinical readout card

5. Wire protocol

Video travels as binary, control as text. Each video message carries a JSON header prefixed with its length, so the protocol stays extensible — the server ignores fields it does not recognise, and no version negotiation is needed.

```
CLIENT ■■■ SERVER    binary, one video frame

[ uint32 LE header_len ][ UTF-8 JSON header ][ raw pixel payload ]

w, h      payload dimensions before rotation
fmt       "nv21" | "rgba" | "jpeg" | "bgra"
rot       0 | 90 | 180 | 270  clockwise, applied server-side
mirror    flip after rotation (selfie view)
mode      UPPER_BODY | LOWER_BODY | FULL_BODY
seq       frame counter, echoed back for round-trip timing

SERVER ■■■ CLIENT    text, one result per processed frame

{ landmarks: { NAME: {x, y, z, v} },    normalised 0..1, upright, mirrored
  angles:    { joint: number | null | string },
  telemetry: { is_ready, guidance_message, confidence, fps, ... },
  physio:    { movement_quality_pct, rom_summary, clinical_feedback },
  seq, track_id, frame_w, frame_h }
```

Why landmarks arrive normalised and pre-mirrored

Rotation and mirroring happen on the server, before inference. The coordinates the client receives are therefore already in the space it is displaying, so the painter applies no correction at all. Alignment between skeleton and video becomes structural rather than something maintained by matching transformations in two places.

Control messages

Message	Effect
set_mode	Switches the body profile; also carried on every frame header so it can never desynchronise from the pixels it applies to
set_filter	One-Euro, Kalman, EMA or none
set_hands	Enables or disables hand tracking on the server — the expensive part
reset_rom	Clears range-of-motion history
screenshot / record	Annotated PNG / MP4 written on the PC via the original renderer

The three-state angle contract

The Python engine deliberately signals three different things that all mean 'no number'. Collapsing them into 0° would be clinically misleading, so the client preserves the distinction end to end through a sealed `AngleState`.

Engine emits	Meaning	Displayed as
<code>null</code>	Landmark confidence below the 0.50 floor	TRACKING...
<code>"SIDE VIEW REQ"</code>	Sagittal shoulder flexion needs a side-on camera	SIDE VIEW REQ
<code>zeroed, is_ready false</code>	Framing invalid — the 'no fake angles' policy	N/A
<code>a number</code>	A real measurement	88.4°

6. Performance engineering

The application initially showed the skeleton trailing the body by two to three seconds. Diagnosis found no single cause but four compounding ones, each modest alone. This section documents them because the reasoning generalises.

6.1 The dominant cost — browser image encoding

The obvious web capture API, `camera_web.takePicture()`, is far heavier than its name suggests. It draws the video to a canvas, encodes JPEG, base64-encodes that into a data URL, wraps it in a Blob, and hands back a file which must then be fetched and base64-decoded. Every frame paid for a compression codec and a base64 round trip — for pixels discarded milliseconds later.

The replacement stops at the canvas. `getImageData` returns pixels directly and raw RGBA goes to the server, which reshapes it with `numpy`.

Per-frame step	<code>takePicture()</code>	Canvas grab
Draw video to canvas	~2 ms	~2 ms
JPEG encode	15–40 ms	—
Base64 encode (<code>toDataURL</code>)	8–20 ms	—
Blob wrap, fetch, base64 decode	10–30 ms	—
<code>getImageData</code>	—	2–4 ms
Total browser cost	35–90 ms	4–6 ms

Trading bandwidth for latency, deliberately

Raw RGBA is about 10× larger on the wire than JPEG — 432 KB versus roughly 40 KB at 384×288. On localhost that costs approximately 8 MB/s, which a loopback socket absorbs without noticing. Spending bandwidth to remove a codec from the critical path is the correct direction when latency is the user-visible quantity.

6.2 The three multipliers

Cause	Why it added lag	Fix
Two frames in flight	A second queued frame never raised throughput — the server processes one at a time — it only guaranteed the frame being analysed was already a full round trip old	Reduced to one; the pipeline always works on the freshest pixels
Glide tied to server interval	Interpolation eased over a whole interval, so a 300 ms stream dragged the skeleton 300 ms behind by design	Capped at 70 ms; past that the skeleton holds the newest known position
Oversized inference frame	Larger frames cost encode time and inference time for accuracy MediaPipe does not use	Inference canvas fixed at 384 px wide, independent of the preview, which stays sharp

6.3 Smooth motion without fabricating data

The server produces roughly 10–20 landmark sets per second; the display refreshes at 60. Painting each set on arrival makes the skeleton visibly step. `PoseInterpolator` keeps the previous and current sets and interpolates between them, sampled from a `Ticker`. The painter receives that ticker as its repaint listenable, so each tick redraws only the skeleton layer inside its `RepaintBoundary` — no widget rebuild. That is what makes 60 fps rendering affordable on top of a 10 fps data stream.

Angles are never interpolated — and that is a clinical decision

Landmark positions are smoothed because they are a picture. Joint angles are not, because they are measurements. A physiotherapist reading 88.4° is entitled to assume the engine computed 88.4° from a frame a camera actually captured. Generating intermediate values would be fabricating clinical data. Smoothing applies to the image, never to the numbers.

6.4 Android capture path

Android avoids the problem entirely. `startImageStream` provides raw YUV420 planes, repacked into NV21 by a stride-aware copy of roughly 2–4 ms, with OpenCV performing colour conversion on the PC in a single SIMD call. The device never touches a codec. Stride handling matters: Android pads plane rows to hardware alignment and delivers chroma as planar or semi-planar depending on the device. Ignoring that is the classic cause of green-skewed or sheared frames; the implementation was verified against both layouts.

7. Interface and rendering

7.1 Camera fit — contain, never cover

An early version fitted the video with `BoxFit.cover` inside a viewport occupying 58% of the screen height. Cover crops to fill: with a 4:3 webcam in a wide, short viewport it scaled to match the width and discarded the remainder. At 1536×730 the camera box was 1536×423 while the frame rendered at 1536×1152 — approximately **37% of the frame was visible**, heavily magnified.

For an assessment that depends on seeing the whole body, cropping is the one thing the fit must never do. It is now `BoxFit.contain`, and the camera occupies the entire window with the readouts floating over it.

Window	cover (before)	contain (now)
1536×730 maximised, 58% split	37% of frame visible	100%
1536×730 maximised, full screen	63%	100%
1100×620 windowed	75%	100%
400×800 phone portrait	67%	100%

7.2 Overlay alignment

The viewport computes the fitted rectangle itself and passes the identical rectangle to both `CameraPreview` and the painter. Alignment is therefore structural: the overlay cannot drift, because both are positioned by the same arithmetic rather than by two transformations kept in agreement.

7.3 Skeleton rendering

The bone tables, arc definitions and visibility floors are a direct port of `medical_gui.py`. Three deliberate changes were made where the original rendering did not survive the move to a large live view.

Change	Reason
Left = teal, right = gold, consistently	The original used blue for the left arm but amber for the left leg, and the reverse for the right. The two brand colours now carry the side cue, so a clinician distinguishes sides at a glance
Face links removed	NOSE–EYE and EYE–EAR sit nearly level on a real face, rendering as a wide V above the nose that reads as antlers. They carry no clinical information — neck inclination derives from NOSE and C7_NECK
Ten facial landmarks dropped	Full-body mode drew every landmark, clustering both eye triples, both ears and both mouth corners into an unreadable blob. NOSE alone anchors the head
Hand skeletons gated on wrist confidence	Below the floor the finger points are extrapolation; drawing them produced a tangle that reads as a malfunction

7.4 Readout layout

Angle cards sit at the four screen corners: left-side joints left, right-side right, upper body up, lower body down. Spatial position carries the meaning, so no label needs parsing to know which limb a number describes. Posture, pelvis and balance are bilateral and sit centre-left, clear of both the corner cards and the mode switcher.

A layout bug worth recording

AngleCard originally used Flexible with a ListView. Inside a Stack with Align, the child receives the full stack height as its maximum — so every card expanded to the entire screen and the bottom card drew straight over the top one. The left card appeared to be missing when it was merely underneath. Cards now size to their content; heights were measured (96 px per limb card, 140 px for posture) and clearance verified at three window sizes.

8. Design system

The Figma brand runs on two accents over a dark ground. That system maps onto clinical semantics cleanly enough that it was used directly rather than having a separate status palette layered on top: **teal means interactive, tracked, in range; gold means attention, end of range, positioning needed.**

Token	Hex	Where it appears
brandTeal	#0E3A46	Splash and header ground, HUD card fill
brandTealDeep	#092B34	Application background
brandTealLight	#14495A	Card headers, inputs, sheets
brandGold	#D9A32B	The wave, right-side limbs, end-of-range warnings
brandGoldLight / Deep	#EFC559 / #A97F1E	Wave gradient edges
brandCyan	#34AFAF	Wordmark, joint nodes, active chips, goniometric arcs
brandCyanLight	#62C4C3	Primary button fill
success	#4FD1A5	Ready state, in-range angles, torso bones
warning	#D9A32B	Brand gold doing exactly its Figma job
danger	#E2664C	Terracotta, warmed to sit beside the gold

Single source of truth

Every colour, radius, gap, stroke width and text style lives in `lib/theme/app_theme.dart`. This was verified programmatically: **no widget file contains a colour literal**. Re-skinning the application is a single-file edit. Each constant is commented with where it appears on screen, so a designer can match Figma layer for layer without navigating the widget tree.

Brand elements as code

The gold wave, watermark and wordmark are drawn with paths and gradients rather than shipped as raster assets, so they stay crisp at any density and recolour instantly. The wave is two layers: gold drawn with a small downward bias, teal clipped to the same curve on top, which produces the gold sliver beneath. Both layers call one shared path builder — if they diverged, a visible seam would appear along the curve.

9. Reliability and failure handling

Backpressure

Latest-frame-wins throughout. The socket refuses to send while a frame is unacknowledged, and the server discards any frame not yet processed when a newer one arrives. Buffering video only converts dropped frames into growing latency, which for a live movement assessment is strictly worse than a lower frame rate — a clinician needs the present, not a complete history. This mirrors the desktop application's `Queue(maxsize=2)` eviction.

Reconnection

Capped exponential backoff at 0.5, 1, 2, 4 then 5 seconds — fast enough to feel instant after a brief network interruption, slow enough not to spam a machine that is switched off. Sessions survive Wi-Fi roaming without the address being retyped.

Distinguishing failure modes

An offline server and a badly framed body both blank the readouts, but the remedies are unrelated. Early on the application showed 'CAMERA POSITIONING — Reconnecting', which reads as a camera fault when the actual cause was the Python server not running. These now have separate banners, colours and copy; the offline banner names the address it is attempting and states plainly that the camera is fine.

Defensive parsing

Every field arriving from the server is parsed tolerantly. Nulls are expected and meaningful, JSON integers are accepted where doubles are expected, NaN and infinity are filtered server-side, and a malformed frame is dropped without disturbing the socket.

Lifecycle

The camera is released when the application is backgrounded — Android revokes it regardless, and holding it produces a black preview on resume. Tickers, timers, stream subscriptions and the wakelock are all released on disposal.

10. Verification

Because the clinical values cross a process boundary, the contract between Dart and Python was verified programmatically rather than by inspection.

Check	Result
Landmark name parity against <code>pose_detector.py</code>	23 body references and 21 hand joints — zero unknown names
Angle key parity against <code>physio_angles.py</code>	17 consumed keys, all produced by the engine — none missing
Wire protocol round trip	Header framing, bounds checks, payload reshape and orientation verified for nv21, rgba and jpeg
NV21 repack	Verified against both padded semi-planar and unpadded planar Android chroma layouts
Colour literals outside the theme	Zero
Structural integrity	All 18 files balanced; every relative import resolves
<code>flutter analyze</code>	No issues found
Unit tests	AngleState three-state contract, PoseFrame parsing, URL normalisation, BodyMode round trip

A lesson about what a clean analyzer does not prove

`flutter analyze` reported no issues while the build still failed: a duplicate element in a `const` set is caught by the compiler front end, not the analyzer. `flutter run` is the real check.

11. Tooling and developer experience

Setup automation was written after each obstacle was hit, so the scripts encode real failures rather than anticipated ones.

Script	Purpose
<code>setup_python.ps1</code>	Finds a Python that can actually import mediapipe and cv2, then builds a venv with <code>--system-site-packages</code> so an existing install is reused rather than re-downloading 300 MB
<code>setup_flutter.ps1</code>	Downloads the Flutter SDK, resolving the current stable release from the official manifest rather than hard-coding a version; sets <code>PATH</code> ; generates <code>web/</code> and <code>android/</code> ; patches the manifest
<code>diagnose.ps1</code>	Prints what one specific terminal resolves and compares its <code>PATH</code> against the stored value
<code>check_setup.ps1</code>	Read-only prerequisite audit
<code>run.bat</code>	Starts the server in place, without spawning an external console
<code>.vscode/tasks.json</code>	A parallel task opening server and app in two integrated terminals

Two bugs in the tooling worth recording

Windows PowerShell 5.1 strips embedded double quotes when passing arguments to native executables, so `python -c 'print("x")'` arrives as a syntax error — which looks exactly like a broken Python install. Every probe now runs from a temporary file. Separately, gating on a version range wrongly rejected a working interpreter: MediaPipe has since shipped wheels for newer Python versions, so any hard-coded range goes stale. Capability is the real question.

12. Known limits and honest trade-offs

Area	Position
Requires the PC server	The application is a client. Nothing clinical happens without the Python engine running
Web frame rate	Chrome is slower than Android because the browser cannot supply raw sensor planes. Android streams YUV420 with no codec at all
Mirroring and side labels	A mirrored frame swaps which limb MediaPipe labels left and right, because the model reasons about the image as presented. Intuitive for a patient watching themselves; disable it when a report needs anatomically literal side labels
Interpolation latency	Rendering moves toward a position already received rather than predicting past it. Extrapolation would overshoot on direction changes and briefly draw limbs where they never went
Sagittal shoulder flexion	Cannot be measured from a front view. The engine returns SIDE VIEW REQ and the application displays it rather than inventing a number
Hand tracking at distance	MediaPipe Hands emits 21 points regardless of confidence. Drawing is gated on wrist confidence; it remains the most expensive stage and can be disabled for throughput
OTP screens not built	The Figma set includes phone and OTP verification, which need an SMS provider and a backend
Wordmark is set type	Not the original letterforms. The swap point is documented in brand.dart

13. Summary

The Flutter application is 6,037 lines of Dart across 18 files, running on Chrome and Android from a single codebase, backed by a 727-line Python server that leaves the clinical engine entirely untouched. Six runtime dependencies, no state-management framework, a design system in one file, and an analyzer that reports no issues.

The engineering that mattered was not the widget tree. It was establishing that the trained model was not in the live path, verifying every landmark and angle key across the process boundary, finding that a browser API was spending 90 ms per frame on work whose output was immediately discarded, and holding the line that a picture may be smoothed while a measurement may not.